Team challenge: improve the neural predictor

The problem in one paragraph

A planner gets one push to land a puck in a green ring after 35 steps. It picks the push by asking a small neural network to predict how the puck moves. The predicted path ends at or very near the target, but when the real simulator runs the same push, the puck often misses. Work out where the prediction goes wrong, change how the network is trained, and show with evidence whether the change helped.

Keep three questions in front of you: **What does the model think will happen? What actually happens? Where do the two differ, and why?** Report mean miss as well as targets reached, because an improvement can show up as a smaller miss before it shows up as a hit.

Bring one laptop per team

Ten teams of six or seven need ten participant laptops. Only the two hosts need ChatGPT Work. Install Python, Jupyter and `student/training-requirements.txt` before the session. CPU training is supported; no API key, paid account or GPU is required. With only two laptops, this becomes a discussion/demo rather than hands-on coding.

What you actually edit

Open `team-notebook.ipynb` from `student/`. It reproduces the original failure, compares true-state one-step prediction with free rollout, diagnoses action selection, trains your candidate and evaluates it.

- `train_repair.py`: neural layers, loss, training data and optimization. This is the editable training starter, not a solved model.
- `neural_model.py`: portable inference from exported neural weights and optional geometry features.
- `repair.py`: the submission interface. `Repair(neural, training_states, training_next_states)` takes batches of `[x,y,vx,vy]` and returns next states with the same shape. It loads your checkpoint, or explicitly reports that it is using the original baseline if none exists.
- `check_submission.py`: fixed development evaluation. Do not change it for scoring.

From this folder:

```
python -m pip install -r training-requirements.txt
python -m jupyterlab team-notebook.ipynb
```

Or train directly:

```
python run_experiment.py --name baseline
python run_experiment.py --name contacts --contact-weight 5 --compare baseline
```

Each experiment is preserved under `results/runs/<name>/` with `network.json`, training report and `score.json`. Load network.json using **Try your team's trained model** in the website to see your network act. Inference stays in that browser tab; training stays on your laptop. Read EXPERIMENTS.md for the exact code locations, hypotheses and workflow.

What to expect

Question	Answer
What is the setup cost?	One install. `training-requirements.txt` includes PyTorch, which is a large download. Do it on good Wi-Fi before the session, then check that `python run_experiment.py --name baseline` runs.
How long does a run take?	Seconds. The starter trains for 40 passes over 24,000 examples on the CPU. In rehearsal that took about 2 to 6 seconds, and a whole run including scoring all 24 scenes took about 15 seconds on a laptop-class CPU. Your laptop may be slower or faster.
What do I get?	A folder `results/runs/<name>/` with `network.json` (your learned weights, about 60 KB), a training report and `score.json`.
Do I upload anything?	No. Training runs only on your laptop. In the website, **Try your team's trained model** reads `network.json` inside your browser tab. Nothing is sent to a server, and it is gone when you close the tab. Give your run folder to the hosts yourselves.
What score should I expect first?	Your baseline scores about **2 out of 24** development scenes in rehearsal. Do not confuse this with the original network shown in the website, which scores 0/24.
What if my change scores 0/24 or worse than the baseline?	That is a normal, useful result. In rehearsal, several sensible-looking single changes made things no better. Keep the run, say what you expected, and explain what the result rules out.
Do I need a GPU, an account or an API key?	No.
What if a run fails?	Run names cannot be reused. If the tool says the run already exists, pick a new `--name`. Do not delete earlier runs. If something else breaks, show a host the error text.
Are the 24 scenes the final test?	No. They are for choosing between your ideas. Hosts score your frozen run once on 96 separate scenes.

Running `python run_experiment.py` uses 40 passes over the data. Running `train_repair.py` directly defaults to 100, so scores from the two are not comparable unless you set the same `--epochs`.

The experiments

1. Reproduce the original network's 128-versus-512 search outcome. Predict before running it.
2. Choose one hypothesis: insufficient contact examples, accumulated free-motion error, insufficient speed coverage, or selection of optimistic predictions. Write a result that would disprove it.
3. Change one factor at a time. Examples: `--contact-weight 5`, `--geometry`, an architectural edit, or `--data expanded`. Preserve previous checkpoints and reports under different names. No improvement is guaranteed by these switches.
4. Run every development scene. Keep candidate pool, 2,048-action budget, 35-step horizon, speed cap 0.11, radius 0.065 and ranking rule fixed. The original network scores 0/24; the trainable starter you begin from scores about 2/24. A good one-step loss alone is insufficient.
5. Identify regressions as well as successes. State which factors changed, rather than attributing a combined change to a single cause.
6. Freeze your code, checkpoint and reports. Each host receives five concise team reports, reviews implementations and evaluates each frozen submission once on 96 separate scenes.

Data and allowed information

`../assets/training-observations.npz` contains the original 30,000 transitions from 1,200 trajectories. The starter splits whole 25-transition trajectories into 960 training and 240 validation trajectories. Normalizers are fit only on the training subset.

`../assets/neural-training.npz` contains 177,319 additional independent training states and 17,713 separate validation states. It includes near-wall and near-bumper states, with speeds up to 0.115. This is an explicitly larger data budget. Original and expanded validation errors describe different distributions and cannot be compared as if the sets were identical. Neither is the final planning test.

Known geometry may be used as neural inputs or to route learned branches if declared. **The change in state must come from learned weights.** Copying simulator responses, substituting a fitted physics model, or querying the simulator during action ranking does not count as a neural repair. Simulator use is allowed for diagnostics and post-selection evaluation. Do not inspect final evaluation scenes or host solutions.

Roles and submission

Operator, experiment designer, skeptic, recorder, and two or three reviewers. Rotate the operator. Every member writes an expected result before the experiment; reviewers inspect assumptions and failures. Submit your chosen run folder (checkpoint, training report, score and team-report.json) together with your edited Python files. Hosts need the folder, not just one pasted prompt.

Scoring: 40 points for hypothesis and distinguishing experiment, 30 for reproducible neural implementation and declared assumptions, 20 for target success with regressions reported, 10 for evidence quality and limitations. A failed but informative experiment can earn investigation points.

This is deterministic, synthetic, single-push planning. It is not a physical robot or a demonstration of general intelligence.